

Assignment No: 4

Student Details

Name: Tanushka Patil

PRN / Roll No: 123B1B036

Class: BTech

Division: A

Subject: DevOps

Title

Create or Migrate an Application to Docker – Containerizing a Node.js Web Application

Aim

To create a simple Node.js web application, run it directly on the host machine, and then migrate it to Docker by writing a Dockerfile, building an image and running the application as a container.

Theory

Why Migrate Applications to Docker?

Traditionally an application is installed directly on a server together with its runtime (Node.js, Python, Java) and libraries. Differences between developer laptops, test servers and production servers then cause the classic “it works on my machine” problem. Docker solves this by packaging the application, its runtime and all dependencies into a portable image that runs identically on any machine with Docker installed.

- **Image:** a read-only, layered template containing the application and everything it needs to run.
- **Container:** a running instance of an image, isolated from other processes by Linux namespaces and cgroups.
- **Dockerfile:** a text file with step-by-step instructions used by docker build to create an image.
- **Registry:** a store for images, e.g. Docker Hub, from which images are pushed and pulled.
- **Layer caching:** each Dockerfile instruction creates a layer; unchanged layers are reused, so builds are fast.

Important Dockerfile Instructions

- **FROM** – base image (here node:20-alpine, a small Linux image with Node.js).
- **WORKDIR** – working directory inside the image.
- **COPY** – copies files from the build context into the image.
- **RUN** – executes a command at build time (npm install).
- **ENV / EXPOSE** – sets environment variables and documents the listening port.
- **USER** – runs the container as a non-root user for better security.
- **HEALTHCHECK** – lets Docker check whether the application is healthy.

- **CMD** – default command executed when the container starts.

Migration Steps

1. Run and test the application on the host (before migration).
2. Write a Dockerfile and a .dockerignore file.
3. Build the image with docker build.
4. Run the container with port mapping and environment variables.
5. Test the containerized application and inspect logs, processes and image layers.
6. Stop and remove the container.

Code / Implementation

Step 1: Application source code

```
Assignment4_Docker/  
├─ package.json  
├─ server.js  
├─ public/  
│   └─ index.html  
├─ Dockerfile  
└─ .dockerignore
```

server.js – Express web server with a REST API

```
const express = require("express");  
const os = require("os");  
  
const app = express();  
const PORT = process.env.PORT || 3000;  
const APP_ENV = process.env.APP_ENV || "local";  
  
app.use(express.json());  
app.use(express.static("public"));  
  
const tasks = [  
  { id: 1, title: "Write Dockerfile", done: true },  
  { id: 2, title: "Build Docker image", done: true },  
  { id: 3, title: "Run container", done: false },  
];  
  
app.get("/api/health", (req, res) => {  
  res.json({ status: "UP", environment: APP_ENV, host: os.hostname() });  
});  
  
app.get("/api/tasks", (req, res) => res.json(tasks));  
  
app.post("/api/tasks", (req, res) => {  
  const task = { id: tasks.length + 1, title: req.body.title, done: false };  
  tasks.push(task);  
  res.status(201).json(task);  
});
```

```
app.listen(PORT, () => {
  console.log(`Student Task Tracker running on port ${PORT} (env: ${APP_ENV})`);
});
```

package.json

```
{
  "name": "student-task-tracker",
  "version": "1.0.0",
  "description": "Simple task tracker web app used for the Docker migration assignment",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

Step 2: Run the application on the host (before migration)

```
npm install
node server.js
curl.exe http://localhost:3000/api/health
```

Step 3: Write the Dockerfile

Dockerfile

```
# ---- Base image ----
FROM node:20-alpine

# Set working directory inside the container
WORKDIR /usr/src/app

# Install dependencies first (better layer caching)
COPY package*.json ./
RUN npm install --omit=dev

# Copy application source
COPY server.js ./
COPY public ./public

# Runtime configuration
ENV PORT=3000 \
    APP_ENV=docker
EXPOSE 3000

# Run as the non-root "node" user
USER node

# Container health check
```

```
HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:3000/api/health || exit 1

CMD ["npm", "start"]
```

.dockerignore – keeps node_modules and Git files out of the image

```
node_modules
npm-debug.log
.git
Dockerfile
.dockerignore
```

Step 4: Build, run and test the container

```
docker build -t student-tasks:1.0 .
docker images student-tasks
docker run -d --rm --name student-tasks -p 8081:3000 student-tasks:1.0
docker ps
curl.exe http://localhost:8081/api/health
curl.exe http://localhost:8081/api/tasks
docker logs student-tasks
docker exec student-tasks sh -c "whoami && node --version"
docker history student-tasks:1.0
docker stop student-tasks
```

Output

1. Application running directly on the host with Node.js (before migration):

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> node --version; npm --version
node v24.12.0
npm 11.6.2

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> npm install

added 68 packages, and audited 69 packages in 32s

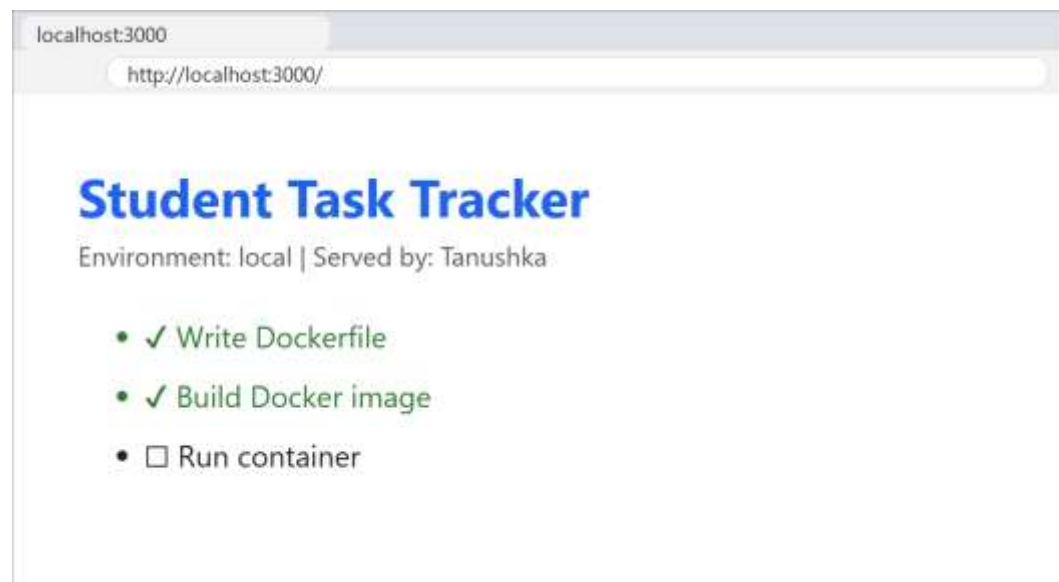
16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> node server.js
Student Task Tracker running on port 3000 (env: local)

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> curl.exe http://localhost:3000/api/health
{"status":"UP","environment":"local","host":"Tanushka"}

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> curl.exe http://localhost:3000/api/tasks
[{"id":1,"title":"Write Dockerfile","done":true},{"id":2,"title":"Build Docker image","done":true}, {"id":3,"title":"Run container","done":false}]
```



2. Building the Docker image from the Dockerfile:

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker --version
Docker version 29.8.0, build 88096ef

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker build -t student-tasks:1.0 .
...
#6 DONE 11.3s
#6 [1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
#6 DONE 11.4s
#7 [2/6] WORKDIR /usr/src/app
#7 DONE 0.4s
#8 [3/6] COPY package*.json ./
#8 DONE 0.1s
#9 [4/6] RUN npm install --omit=dev
#9 DONE 7.4s
#10 [5/6] COPY server.js ./
#10 DONE 0.2s
#11 [6/6] COPY public ./public
#11 DONE 0.1s
#12 DONE 3.8s

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker images student-tasks
IMAGE          ID                                DISK USAGE  CONTENT SIZE  EXTRA
student-tasks:1.0  1f4817221e9d                    202MB       49.9MB
```

3. Running the container (host port 8081 → container port 3000) and testing the REST API:

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker run -d --rm --name student-tasks -p 8081:3000 student-tasks:1.0
c897073fda382540d220df96b5e278839dce074a6872459a8a85c4c842471059

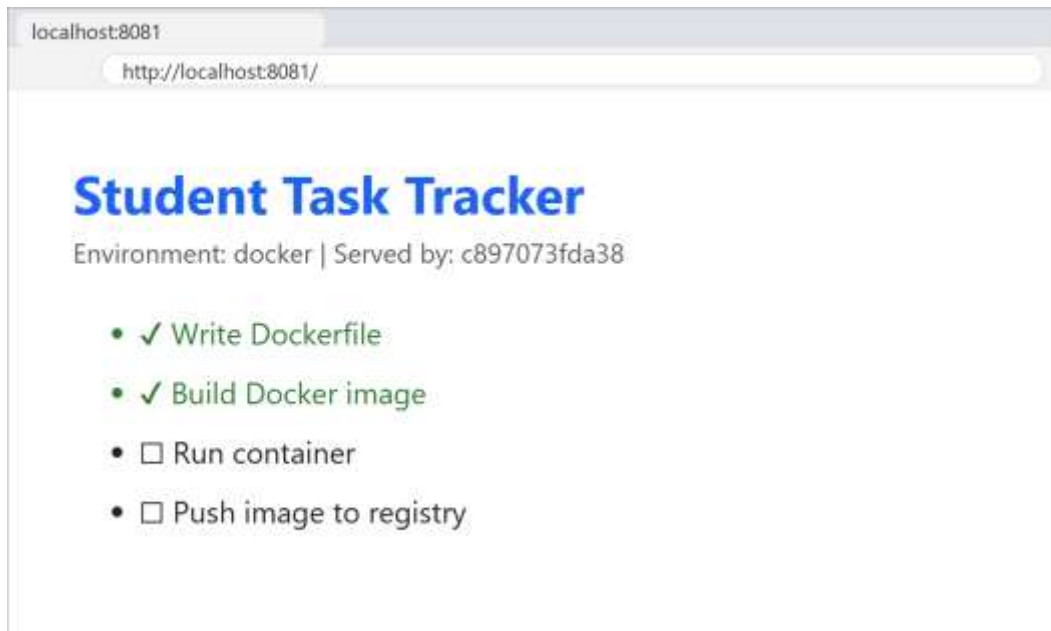
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker ps --filter name=student-tasks
NAMES          IMAGE          STATUS          PORTS
student-tasks  student-tasks:1.0  Up 7 seconds (health: starting)  0.0.0.0:8081->3000/tcp, [::]:8081->3000/tcp

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> curl.exe http://localhost:8081/api/health
{"status":"UP","environment":"docker","host":"c897073fda38"}

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> curl.exe http://localhost:8081/api/tasks
[{"id":1,"title":"Write Dockerfile","done":true},{"id":2,"title":"Build Docker image","done":true},{"id":3,"title":"Run container","done":false}]

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> curl.exe -X POST -H "Content-Type: application/json" -d '{"title":"Push image to registry"}' http://localhost:8081/api/tasks
{"id":4,"title":"Push image to registry","done":false}
```

4. The same application now served from the container – note the environment is docker and the host is the container ID:



5. Container logs, running as the non-root node user, and health status:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker logs student-tasks

> student-task-tracker@1.0.0 start
> node server.js

Student Task Tracker running on port 3000 (env: docker)

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker exec student-tasks sh -c "whoami && node --version && ls -la /usr/src/app"
node
v20.20.2
total 56
drwxr-xr-x  1 root    root      4096 Sep 29 17:48 .
drwxr-xr-x  1 root    root      4096 Sep 29 17:48 ..
drwxr-xr-x 70 root    root      4096 Sep 29 17:48 node_modules
-rwxr-xr-x  1 root    root     29490 Sep 29 17:48 package-lock.json
-rwxr-xr-x  1 root    root      271 Sep 29 17:46 package.json
drwxr-xr-x  2 root    root      4096 Sep 29 17:46 public
-rwxr-xr-x  1 root    root      864 Sep 29 17:46 server.js

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker ps --filter name=student-tasks (after health check)
NAME                STATUS
student-tasks       Up 40 seconds (healthy)
```

6. Image layers created by the Dockerfile instructions:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker history student-tasks:1.0
CREATED BY          SIZE
CMD ["npm" "start"] 0B
HEALTHCHECK {Test:[CMD-SHELL wget -qO- http:... 0B
USER node           0B
EXPOSE [3000/tcp]   0B
ENV PORT=3000 APP_ENV=docker 0B
COPY public ./public # buildkit 24.6kB
COPY server.js ./ # buildkit 20.5kB
RUN /bin/sh -c npm install --omit=dev # buil... 6.99MB
COPY package*.json ./ # buildkit 53.2kB
WORKDIR /usr/src/app 16.4kB
CMD ["node"]        0B
...
```

7. Stopping the container (started with --rm, so it is removed automatically):

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker stop student-tasks
student-tasks

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment4_Docker> docker ps -a --filter name=student-tasks

CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
```

Conclusion

In this assignment, I learned about migrating an application to Docker. I first ran a Node.js web application directly on the host, then wrote a Dockerfile using FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, USER, HEALTHCHECK and CMD, built an image, and ran the same application as a container with port mapping. The containerized app behaved exactly like the original but no longer depends on software installed on the host, which makes it portable and consistent across environments.

References

1. Dockerfile Reference – <https://docs.docker.com/reference/dockerfile/>
2. Docker Build Overview – <https://docs.docker.com/build/concepts/overview/>
3. Containerize a Node.js Application – <https://docs.docker.com/guides/nodejs/>
4. Node.js Official Docker Image – https://hub.docker.com/_/node
5. Express.js Documentation – <https://expressjs.com/>

Submission Links

GitHub Repository Link:

https://github.com/tanushkapatil/DevOps/tree/main/Assignment4_Docker

Hosted / Output Link: <http://localhost:8081> (local output – container stopped after testing)